

Introdução à Programação orientada a objetos

Marcos Monteiro Junior

Programando com Java

Sumário

1	Fundamentos de Orientação a Objetos	3
1.1	Encapsulamento	3
1.1.1	Modificadores de acesso	3
1.2	Métodos e atributos estáticos	5
1.2.1	Getters e Setters	5
1.2.2	Construtores	7
1.2.3	Sobrecarga e assinatura de métodos	8
1.2.4	Método <code>toString</code>	9
1.2.5	Método <code>equals</code>	9
1.2.6	Atributos estáticos	10
	Referências Bibliográficas	12

1 Fundamentos de Orientação a Objetos

1.1 Encapsulamento

O encapsulamento é um dos conceitos mais utilizados na Programação Orientada a Objetos. A ideia consiste em ocultar informações e a complexidade interna de um objeto, disponibilizando apenas o necessário para quem o utiliza. Dessa forma, é possível blindar as características internas do objeto em relação a outras classes, evitando resultados inesperados, acessos indevidos e outros problemas de integridade. Nessa seção vamos ver algumas formas de fazer encapsulamento.

1.1.1 Modificadores de acesso

Em Java o encapsulamento pode ser feito com os modificadores de acesso. Os modificadores são palavras que dizem ao sistema quais informações podem ser acessadas fora da classe ou por classes que pertencem ao mesmo pacote da classe atual. Os modificadores de acesso em Java são:

- *private*: O nível máximo de encapsulamento. O membro só é visível dentro da própria classe. É aqui onde os atributos ficam protegidos.
- *protect*: Permite acesso dentro do mesmo pacote e por subclasses (herança).
- *public*: O nível mínimo de encapsulamento. Qualquer classe pode acessar o membro. Usado para os métodos que formam a “interface” pública do objeto.
- *default* (sem modificador): Acesso restrito ao pacote onde a classe reside.

Portanto, é prática comum definir atributos com o modificador de acesso *private* e métodos como *public*, embora essa não seja uma regra rígida, mas sim uma convenção de segurança. Um exemplo clássico para ilustrar a aplicação desses modificadores é a classe *ContaBancaria*. O atributo *saldo*, por exemplo, não deve ser modificado diretamente por outras classes, pois isso permitiria a manipulação arbitrária de valores, ferindo a integridade dos dados. Para impedir essa situação, declaramos o atributo *saldo* como *private*. Dessa forma, qualquer alteração externa deve ocorrer obrigatoriamente por meio de métodos *public*, os quais podem implementar as validações necessárias para assegurar as regras de negócio do sistema. Observe o exemplo abaixo:

```
public class ContaBancaria {  
    // Encapsulamos o atributo para proteger a integridade  
    private double saldo;
```

```
// Fornecemos acesso controlado via metodos (Interface Publica
)
public void depositar(double valor) {
    if (valor > 0) {
        this.saldo += valor;
    }
}
public double getSaldo() {
    return this.saldo;
}
}
```

Voltando ao nosso exemplo da clínica veterinária, vamos ajustar os modificadores de acesso dos atributos e métodos para aplicar o encapsulamento. Adicionaremos à classe `Pet` os métodos `registrarPet()`, `listarPet()` e `buscarPet()`, que compõem operações fundamentais do padrão CRUD (*Create* - Criar, *Read* - Ler, *Update* - Atualizar e *Delete* - Deletar). Por enquanto, estes métodos exibirão apenas mensagens de console, simulando a execução das ações que serão implementadas detalhadamente nos próximos capítulos.

```
public class Pet {
    // Atributos com encapsulamento (private)
    private String nomeAnimal;
    private int idadeAnimal;
    private int sexoAnimal;
    private boolean vacinado;

    public void vacinar() {
        this.vacinado = true;
        System.out.println(this.nome + " foi vacinado com sucesso!
        ");
    }
    // Metodos (CRUD) com implementacao simplificada (stubs)
    public void registrarPet() {
        System.out.println("Acao: Registrando novo pet no sistema
        ...");
    }

    public void listarPet() {
        System.out.println("Acao: Listando pets cadastrados...");
    }

    public void buscarPet() {
        System.out.println("Acao: Buscando pet no banco de dados
        ...");
    }
}
```

Dessa forma outras classes só terão acesso aos métodos públicos, enquanto os atributos permanecem protegidos, garantindo a integridade e consistência dos dados. Na próxima

seção vamos ver como acessar os atributos de forma controlada, utilizando os métodos getters e setters.

1.2 Métodos e atributos estáticos

Nessa seção será apresentado o conceito de métodos comumente utilizados e atributos estáticos.

1.2.1 Getters e Setters

Com a aplicação de modificadores de acesso restritivos (como o `private`), não é possível acessar diretamente os atributos de um objeto a partir de outras classes. Na prática, contudo, precisamos manipular esses dados de forma controlada; para isso, utilizamos métodos específicos. Por convenção, os métodos destinados a retornar o valor de um atributo iniciam com o prefixo *get*, enquanto aqueles destinados a alterar seu conteúdo utilizam o prefixo *set* em sua nomenclatura. Veja o exemplo para nossa classe `Pet`.

```
public class Pet {  
    // Atributos com encapsulamento (private)  
    private String nomeAnimal;  
    private int idadeAnimal;  
    private int sexoAnimal;  
    private boolean vacinado;  
  
    public String getNomeAnimal() {  
        return nomeAnimal;  
    }  
    public void setNomeAnimal(String nomeAnimal) {  
        this.nomeAnimal = nomeAnimal;  
    }  
    public int getIdadeAnimal() {  
        return idadeAnimal;  
    }  
    public void setIdadeAnimal(int idadeAnimal) {  
        this.idadeAnimal = idadeAnimal;  
    }  
    public int getSexoAnimal() {  
        return sexoAnimal;  
    }  
    public void setSexoAnimal(int sexoAnimal) {  
        this.sexoAnimal = sexoAnimal;  
    }  
    public boolean isVacinado() {  
        return vacinado;  
    }  
    public void vacinar() {  
        this.vacinado = true;  
    }  
}
```

```
        System.out.println(this.nome + " foi vacinado com sucesso!"
            + "\n");
    }
    // Metodos (CRUD) com implementacao simplificada (stubs)
    public void registrarPet() {
        System.out.println("Acao: Registrando novo pet no sistema
            ...");
    }
    public void listarPet() {
        System.out.println("Acao: Listando pets cadastrados...");
    }

    public void buscarPet() {
        System.out.println("Acao: Buscando pet no banco de dados
            ...");
    }
}
```

Observação

É uma má prática criar uma classe e, logo em seguida, criar getters e setters para todos seus atributos. Você só deve criar um getter ou setter se tiver a real necessidade.

Outras vantagens do uso de getters e setters incluem:

- Validação de Atributos: No método setter, você pode adicionar regras de validação antes de atribuir o valor. Por exemplo, garantir que a idade não seja um número negativo ou que um saldo bancário não receba valores inválidos.
- Controle de Acesso (Leitura ou Escrita): Você pode criar propriedades somente leitura (criando apenas o getter e omitindo o setter) ou somente escrita, controlando exatamente como os dados são expostos.
- Flexibilidade e Manutenibilidade: Se você precisar mudar a forma como um dado é armazenado ou calculado internamente, basta alterar o código dentro do getter/setter. O restante do sistema que utiliza esses métodos não precisará ser reescrito, pois a interface externa continua a mesma.

Voltando ao nosso exemplo, agora vamos modificar nossa classe principal, para ver os métodos públicos getters e setters.

```
public class Principal {
    public static void main(String[] args) {
        // 1. Instanciacao do objeto
        Pet meuPet = new Pet();

        // 2. Uso dos metodos SET para definir os atributos
    }
}
```

```
meuPet.setNomeAnimal("Rex");
meuPet.setIdadeAnimal(3);
meuPet.setSexoAnimal(1); // Ex: 1 para macho, 2 para fema

// 3. Uso dos metodos CRUD
meuPet.registrarPet();

// 4. Acao especifica: vacinar
System.out.println("Status atual antes da vacina: " +
    meuPet.isVacinado());
meuPet.vacinar();

// 5. Uso dos metodos GET para exibir informacoes
System.out.println("Nome do Pet: " + meuPet.getNomeAnimal
    ());
System.out.println("Idade do Pet: " + meuPet.
    getIdadeAnimal());
System.out.println("Status apos vacina: " + meuPet.
    isVacinado());

// 6. Outros metodos de simulacao
meuPet.listarPet();
meuPet.buscarPet();
}
}
```

1.2.2 Construtores

Quando usamos o comando **new**, estamos construindo um objeto e ele chama o construtor da classe. O construtor da classe é semelhante a um método, apesar da semelhança ele não é considerado um método propriamente dito. Para criar um construtor utilizamos **mesmo nome da classe** (observe o exemplo abaixo)(Cabral (2014)).

```
...
    public Pet(){ }
...
```

Em nenhum exemplo dessa apostila até então foi criado um construtor. Então como o objeto foi criado na classe principal? Se não existir esse método, Java cria ele automaticamente, chamamos isso de *construtor default*. Um construtor default não possui argumentos (parâmetros), ou seja o objeto inicia com os atributos nulos (NULL).

O uso de um construtor se justifica em várias situações, como, por exemplo, desejar-se que os objetos sejam iniciado pelo usuário com valores que são obrigatórios. Veja o exemplo abaixo.

```
...
    public Pet(String nome){
        this.nomeAnimal = nome;
    }
}
```

```
}
```

Nesse exemplo, obrigamos a criação do objeto Pet com nome do animal. A ainda uma situação onde um objeto só faz sentido existir se outro objeto existir (será discutido isso no próximo capítulo), nesse caso precisamos passar como argumento esse objeto.

1.2.3 Sobrecarga e assinatura de métodos

Em Java, a definição de um comportamento não se resume apenas ao nome que damos ao método. Para o compilador, a identidade de uma ação é definida pela sua assinatura.

Métodos não são definidos apenas pelo nome: O nome é apenas o identificador primário. Para diferenciar funcionalidades, o Java utiliza um conjunto de características chamado assinatura.

Assinatura de um método define o seu “nome completo”: É a assinatura que permite ao compilador saber exatamente qual bloco de código deve ser executado quando um método é chamado.

Fazem parte da assinatura de um método:

- Nome do método: A identificação básica.
- Quantidade de parâmetros: Quantos dados o método recebe.
- Tipos dos parâmetros: Quais são os tipos de dados (ex: int, String, float).
- Ordem dos parâmetros: A sequência em que os tipos são declarados.

São métodos diferentes, apesar do mesmo nome: Quando mantemos o nome, mas alteramos os parâmetros, criamos variações da mesma ação:

```
public void metodo()  
  
public void metodo(int a)  
  
public void metodo(float a)  
  
public void metodo(int a, float b)  
  
public void metodo(float a, int b)
```

Essa característica da programação orientada a objetos damos o nome de sobrecarga ou *Overloading*. Ela oferece flexibilidade e legibilidade, permitindo que o desenvolvedor utilize o mesmo nome para operações que, embora semanticamente similares, aceitem entradas de dados diferentes.

Logo, podemos ter vários construtores, com diferentes assinaturas utilizando essa propriedade.

Observação

O **overloading** é uma característica da programação orientada a objetos que permite a criação de múltiplos métodos com o mesmo nome, mas com diferentes assinaturas.

1.2.4 Método toString

O método `toString()` é um componente fundamental da classe `Object` em Java. Por padrão, quando tentamos exibir um objeto diretamente, o Java retorna um código hexadecimal pouco compreensível (como `Pet@15db9742`). Portanto, o método `toString()` é extremamente útil para exibir as informações de um objeto de forma legível para humanos. Para que essa representação seja útil no nosso sistema de pets, podemos sobrescrever esse método em nossas classes personalizadas, garantindo que ele retorne os dados que realmente importam.

```
public class Pet {
    private String nomeAnimal;
    private int idadeAnimal;
    private int sexoAnimal;

    public Pet(String nome, int idade, int sexo) {
        this.nomeAnimal = nome;
        this.idadeAnimal = idade;
        this.sexoAnimal = sexo;
    }
    // Sobrescrita do metodo, abordaremos no proximo capitulo
    @Override
    public String toString() {
        return "Pet [Nome: " + nomeAnimal + ", Idade: " +
            idadeAnimal + " anos, " + Sexo: " + sexoAnimal]";
    }
}

// No uso do sistema:
Pet meuPet = new Pet("Rex", 3);
System.out.println(meuPet);
// Saída no console: Pet [Nome: Rex, Idade: 3 anos, Sexo: 1]
```

1.2.5 Método equals

Como descrito no início do capítulo, a identidade do objeto é única e imutável, portanto, é muito difícil de comparar dois objetos, mesmo que eles possuam os mesmos atributos e valores. Para isso, o Java fornece o método `equals()` que compara se dois objetos são iguais, ou seja, se possuem os mesmos valores de atributos.

1 Fundamentos de Orientação a Objetos

Para utilizar esse método, precisamos, sobrescrever o método `equals()` na classe que desejamos comparar. Abaixo segue um exemplo de como implementar o método `equals()` na classe `Pet`.

```
public class Pet {
    private String nomeAnimal;
    private int idadeAnimal;
    private int sexoAnimal;
    private boolean vacinado;

    // Construtor e outros metodos omitidos

    @Override
    public boolean equals(Object obj) {
        if (obj instanceof Pet) {
            Pet p = (Pet) obj;
            return this.nomeAnimal.equals(p.nomeAnimal) &&
                this.idadeAnimal == p.idadeAnimal &&
                this.sexoAnimal == p.sexoAnimal;
        }
        return false;
    }
}
```

Observação

O operador `instanceof` em Java é uma palavra-chave utilizada para realizar testes de tipo em tempo de execução (*runtime type checking*). Ele permite verificar se um objeto é uma instância de uma classe específica, se implementa uma determinada interface ou se pertence a uma subclasse.

1.2.6 Atributos estáticos

Até agora nesse capítulo utilizamos métodos e atributos de instância, ou seja, cada objeto possui seus próprios valores. No entanto, existem situações em que queremos que um atributo ou método seja compartilhado entre todos os objetos de uma classe. Para isso, utilizamos o modificador `static`. Atributos e métodos estáticos pertencem à classe em si, e não a instâncias individuais. Isso significa que eles podem ser acessados diretamente através do nome da classe, sem a necessidade de criar um objeto.

No exemplo da clinica, não faz muito sentido o uso de atributos estáticos, mas um exemplo seria manter o números de pets cadastrados.

```
...
    private static int contadorPets = 0;
...
```

Observação

Por isso, eles só podem acessar outros membros estáticos da mesma classe. Essa restrição existe porque, ao invocar um método estático, não há uma instância associada — ou seja, não temos acesso à referência *this*. Como o *this* representa o objeto em execução e métodos estáticos são chamados diretamente via nome da classe, torna-se impossível acessar dados que dependeriam de um objeto particular para existir (Cabral (2014)).

Um novo exemplo de uso de atributos estáticos é a classe **Math** do Java, que contém métodos e constantes matemáticas. Por exemplo, a constante **Math.PI** representa o valor de pi e é acessada diretamente pela classe, sem a necessidade de criar uma instância da classe **Math**. Veja o exemplo abaixo:

```
public class ExemploMath {  
    public static void main(String[] args) {  
        double raio = 5.0;  
        double area = Math.PI * Math.pow(raio, 2); // nao e  
            necessario criar um objeto da classe Math para acessar  
            a constante PI e o metodo pow()  
        System.out.println("Area do circulo: " + area);  
    }  
}
```

Referências Bibliográficas

Cabral, L. P. (2014). Java e orientação a objetos.